

Applied Machine Learning Practice Set

Comprehensive Step-by-Step Solutions

Prof. Saurabh

March 8, 2026

Contents

1	ML Fundamentals & Data Preprocessing	2
2	Model Evaluation & Bias-Variance Tradeoff	6
3	Linear Regression & Optimization	9
4	Classification Algorithms (Naive Bayes & Logistic Regression)	12

1 ML Fundamentals & Data Preprocessing

Question 1.1: Conceptual: Categorizing ML Scenarios

Categorize the following machine learning scenarios into their respective types of learning (Supervised, Unsupervised, or Reinforcement Learning). Briefly justify your choice for each.

- (a) An algorithm analyzing a massive unannotated database of medical records to discover hidden, naturally occurring subgroups of patients with similar symptom profiles.
- (b) A drone software that learns to fly through a dense forest by receiving a +50 reward for safely passing trees and a -100 penalty for crashing.
- (c) A model forecasting tomorrow's exact temperature in Celsius using historical weather data containing temperature, humidity, and wind speed labels.

Solution

- (a) **Unsupervised Learning:** The algorithm is tasked with finding hidden patterns or structures (subgroups/clusters) in data that is unannotated and lacks predefined labels.
- (b) **Reinforcement Learning:** The drone acts as an agent learning to make sequential decisions in an environment (the forest) through trial and error, aiming to maximize a cumulative reward signal (+50 for success, -100 for crashing).
- (c) **Supervised Learning (Specifically, Regression):** The model learns from labeled training data (historical weather with known continuous temperature outputs) to predict a continuous numerical value (tomorrow's temperature).

Question 1.2: Case-based: Pipeline & Data Quality Challenges

You are hired as an ML engineer for a manufacturing plant to predict machine downtime based on sensor data.

- (a) Outline the first four steps of an end-to-end machine learning pipeline for this problem.
- (b) Describe two specific data-quality challenges you might discover in this real-world sensor dataset and briefly state how each would negatively impact your model if not fixed.

Solution

- (a) **First Four Pipeline Steps:**
 1. **Data Collection & Ingestion:** Gathering raw telemetry data (vibration, heat, operating hours) from factory machines.
 2. **Data Preprocessing & Cleaning:** Handling missing sensor readings, removing duplicate logs, and filtering out erroneous extreme spikes.

3. **Feature Engineering & Selection:** Creating new features (e.g., rolling average of heat over 24 hours) and selecting the most relevant variables.
 4. **Model Selection & Training:** Choosing a classification algorithm (e.g., Logistic Regression) to predict "Downtime" vs "Running" and training it.
- (b) **Data Quality Challenges:**
- **Irregular Sampling / Missing Values:** Sensors might drop connections, leading to gaps in time-series data. *Impact:* Models generally cannot process null values; it would cause training to fail or introduce severe bias if zeros are incorrectly assumed.
 - **Uncalibrated Scales (Noise/Outliers):** One sensor might measure temperature in Celsius while an older one uses Fahrenheit, or a glitch might record a vibration of 9999. *Impact:* Algorithms sensitive to scale (like KNN or gradient descent-based models) will heavily overweight the larger numbers, ruining predictive reliability.

Question 1.3: Numerical: Feature Scaling

A dataset for predicting house prices contains a feature representing "House Age" (in years) with the following observed values: 5, 10, 25.

- (a) Apply Min-Max Scaling to normalize the value 10 to a fixed range of [0, 1].
- (b) Calculate the Z-score normalized value for the age 25. Assume this small dataset represents the entire population for calculating the standard deviation.

Solution

(a) **Min-Max Scaling for $X = 10$**

- **Formula:** $X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$
- **Parameters:** $X_{min} = 5$, $X_{max} = 25$, $X = 10$
- **Calculation:** $X_{scaled} = \frac{10 - 5}{25 - 5} = \frac{5}{20} = 0.25$

(b) **Z-Score Normalization for $X = 25$**

- **Formula:** $Z = \frac{X - \mu}{\sigma}$
- **Calculate Mean (μ):** $\mu = \frac{5 + 10 + 25}{3} = \frac{40}{3} \approx 13.333$
- **Calculate Population Variance (σ^2):**

$$\sigma^2 = \frac{(5 - 13.333)^2 + (10 - 13.333)^2 + (25 - 13.333)^2}{3}$$

$$\sigma^2 = \frac{(-8.333)^2 + (-3.333)^2 + (11.667)^2}{3} = \frac{69.44 + 11.11 + 136.12}{3} = \frac{216.67}{3} \approx 72.22$$

- **Calculate Standard Deviation (σ):** $\sigma = \sqrt{72.22} \approx 8.50$
- **Calculation for $X = 25$:** $Z = \frac{25 - 13.333}{8.50} = \frac{11.667}{8.50} \approx 1.37$

Question 1.4: Conceptual: Categorical Encoding Strategies

You are working with a dataset to predict user churn. The dataset contains two categorical features: "Subscription Tier" (Free, Basic, Premium) and "Payment Method" (Credit Card, PayPal, Bank Transfer). Explain which encoding technique (One-Hot Encoding or Label Encoding) you would apply to each feature and why. Provide a brief illustration of the output for each.

Solution

1. "Subscription Tier" (Ordinal Data)

- **Technique: Label Encoding.**
- **Why:** This feature has a natural, inherent mathematical order (Free < Basic < Premium). Label encoding captures this relationship, allowing the model to understand that one tier is "higher" than another.
- **Illustration Output:** Free = 0, Basic = 1, Premium = 2.

2. "Payment Method" (Nominal Data)

- **Technique: One-Hot Encoding.**
- **Why:** There is no logical ordering or ranking between Card, PayPal, and Bank Transfer. If we used Label Encoding (e.g., Card=0, PayPal=1, Bank=2), the model might falsely interpret Bank Transfer as being "twice as great" as PayPal, which is mathematically nonsensical. One-Hot encoding prevents this.
- **Illustration Output:** Creates 3 new columns: `Is_Card`, `Is_PayPal`, and `Is_Bank`. A PayPal user would be encoded as [0, 1, 0].

Question 1.5: Numerical: Outlier Detection using IQR

A dataset tracking the daily temperature (in Fahrenheit) over a week contains the following values: 68, 70, 72, 74, 76, 78, 105. Using the Interquartile Range (IQR) method, mathematically determine if the value 105 is considered an outlier. Show all steps involving the five-number summary.

Solution

Step 1: Determine the Quartiles. For the sorted data {68, 70, 72, 74, 76, 78, 105} (n=7):

- **Median (Q2):** The middle value is 74.
- **Q1 (First Quartile):** The median of the lower half {68, 70, 72} is 70.
- **Q3 (Third Quartile):** The median of the upper half {76, 78, 105} is 78.

Step 2: Calculate the Interquartile Range (IQR).

$$IQR = Q3 - Q1 = 78 - 70 = 8$$

Step 3: Calculate Outlier Thresholds. Standard outlier boundaries are defined as

$1.5 \times IQR$ beyond the quartiles:

- Lower Bound = $Q1 - (1.5 \times IQR) = 70 - (1.5 \times 8) = 70 - 12 = \mathbf{58}$
- Upper Bound = $Q3 + (1.5 \times IQR) = 78 + (1.5 \times 8) = 78 + 12 = \mathbf{90}$

Conclusion: Any temperature below 58 or above 90 is considered an outlier. Since $105 > 90$, the value 105 is **mathematically confirmed as an outlier**.

2 Model Evaluation & Bias-Variance Tradeoff

Question 2.1: Numerical: Confusion Matrix Metrics

A fraud detection model evaluated 500 bank transactions. The resulting confusion matrix is as follows:

- True Positive (Fraud predicted as Fraud) = 15
- False Positive (Normal predicted as Fraud) = 10
- False Negative (Fraud predicted as Normal) = 5
- True Negative (Normal predicted as Normal) = 470

Calculate the Precision, Recall, and F1-Score for the Fraud class.

Solution

Step 1: Precision Precision measures the proportion of actual positives among all predicted positive cases.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{15}{15 + 10} = \frac{15}{25} = \mathbf{0.60} \text{ (or 60\%)}$$

Step 2: Recall (Sensitivity) Recall measures the proportion of actual positive cases that were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{15}{15 + 5} = \frac{15}{20} = \mathbf{0.75} \text{ (or 75\%)}$$

Step 3: F1-Score The F1-Score is the harmonic mean of Precision and Recall, providing a balanced metric.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.60 \times 0.75}{0.60 + 0.75} = 2 \times \frac{0.45}{1.35} \approx \mathbf{0.667}$$

Question 2.2: Conceptual: Accuracy Paradox & Imbalanced Data

A hospital develops a machine learning model to predict a rare genetic disorder that occurs in 0.5% of the population. They deploy a baseline model that simply predicts "No Disorder" for every single patient.

- What is the accuracy of this baseline model?
- Explain why relying solely on accuracy to evaluate models in this scenario is dangerous, and suggest two better evaluation metrics they should use instead.

Solution

- Baseline Model Accuracy** Since 99.5% of the population does not have the disorder, and the model always predicts "No Disorder", it will be correct exactly **99.5%** of the time.
- The Danger of Accuracy & Better Metrics** This scenario suffers from a severe

class imbalance. Relying on raw accuracy is highly misleading (the Accuracy Paradox) because an ostensibly "excellent" 99.5% accuracy masks the fact that the model is completely useless at its primary job: finding sick patients. The model never detects the minority class, meaning its Recall for the disorder is 0%.

Better Metrics to Use:

- **Recall (Sensitivity):** Crucial in healthcare to measure the percentage of actual sick patients the model correctly identifies.
- **F1-Score:** Provides a balanced measure between Precision and Recall, ensuring the model is penalized if it ignores the minority class entirely.

Question 2.3: Conceptual: Bias-Variance Diagnosis

While evaluating a linear prediction model, you plot its learning curves and notice that the model exhibits both a very high training error and a very high validation error.

- (a) Using the concepts of Bias and Variance, explain what this indicates about your model's current state.
- (b) If you were to apply a heavy L1 Regularization (Lasso) penalty to this specific model, would it help or worsen the situation? Analytically justify your answer.

Solution

(a) Diagnosing Model State A model exhibiting both high training error and high validation error is suffering from **Underfitting (High Bias)**. This indicates that the model is too simple to capture the underlying patterns or trends in the data. It makes strong, incorrect assumptions about the data's form, leading to poor performance across all datasets.

(b) Impact of L1 Regularization Applying a heavy L1 Regularization (Lasso) penalty would **worsen the situation**. **Justification:** Regularization is designed to prevent overfitting by penalizing model complexity and pulling the model towards simplicity. It does this by adding a penalty term that drives coefficients to zero. Because the current model is already underfitting (too simple), forcing it to be even simpler by shrinking or eliminating its parameters will increase the bias further, making the training and validation errors even higher.

Question 2.4: Numerical: Regression Error Metrics

You are evaluating a regression model that predicts the delivery time (in days) for online orders. For a test set of 4 orders, the actual delivery times (y) and predicted delivery times ($\hat{y}$) are: Actual (y): [3, 5, 2, 8] Predicted ($\hat{y}$): [4, 5, 4, 6] Calculate the Mean Absolute Error (MAE) and the Root Mean Squared Error (RMSE).

Solution

Step 1: Compute Individual Errors ($e_i = y_i - \hat{y}_i$).

- $e_1 = 3 - 4 = -1$
- $e_2 = 5 - 5 = 0$
- $e_3 = 2 - 4 = -2$
- $e_4 = 8 - 6 = 2$

Step 2: Calculate Mean Absolute Error (MAE). Formula: $MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$

$$MAE = \frac{|-1| + |0| + |-2| + |2|}{4} = \frac{1 + 0 + 2 + 2}{4} = \frac{5}{4} = \mathbf{1.25 \text{ days}}$$

Step 3: Calculate Root Mean Squared Error (RMSE). Formula: $RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$

$$MSE = \frac{(-1)^2 + 0^2 + (-2)^2 + (2)^2}{4} = \frac{1 + 0 + 4 + 4}{4} = \frac{9}{4} = 2.25$$

$$RMSE = \sqrt{2.25} = \mathbf{1.5 \text{ days}}$$

Question 2.5: Conceptual: Data Splitting Validation

Explain the distinct roles of the Training Set, Validation Set, and Test Set in a standard machine learning workflow. Furthermore, briefly explain the mechanics of K-Fold Cross-Validation.

Solution

Distinct Roles:

- **Training Set (60-80%):** Used exclusively to train the model. The algorithm learns its internal parameters (weights and biases) directly from this data.
- **Validation Set (10-20%):** Used during the development phase to evaluate the model's performance while tuning hyperparameters (e.g., learning rate, polynomial degree) and to detect/prevent overfitting early.
- **Test Set (10-20%):** Locked away and strictly withheld until the very end of the project. It is used *only once* to provide a final, unbiased estimate of how the model will perform in the real world.

Mechanics of K-Fold Cross-Validation: In K-Fold Cross-Validation, the original dataset is partitioned into K equal subsets (folds). The training process iterates K times. In each iteration, one specific fold is held out as the validation set, while the model is trained on the remaining $K - 1$ folds combined. A performance score is calculated for each iteration, and the final model performance is the average of these K scores. This ensures maximum data utilization and reliable performance estimates.

3 Linear Regression & Optimization

Question 3.1: Numerical: Multiple Linear Regression Prediction

A multiple linear regression model predicts crop yield (in kg) based on the amount of Fertilizer applied (in kg) and the count of Pests observed. The learned equation is:

$$\text{Yield} = 1500 + 40 \times \text{Fertilizer} - 15 \times \text{Pests}$$

- (a) Interpret the coefficient "-15".
- (b) Calculate the predicted crop yield for a farm that applies 20 kg of fertilizer and observes 30 pests.

Solution

(a) Coefficient Interpretation The coefficient "-15" associated with the Pests feature indicates that for every additional 1 unit increase in the pest count, the predicted crop yield *decreases* by 15 kg, assuming the amount of fertilizer is held constant.

(b) Prediction Calculation Substitute the given values into the regression equation:

- Fertilizer (x_1) = 20
- Pests (x_2) = 30

$$\text{Yield} = 1500 + (40 \times 20) - (15 \times 30)$$

$$\text{Yield} = 1500 + 800 - 450$$

$$\text{Yield} = 2300 - 450 = \mathbf{1850 \text{ kg}}$$

Question 3.2: Numerical: Gradient Descent Iterations

Assume a cost function $J(\theta) = 3\theta^2 + 5$. We want to minimize this function using Gradient Descent. Given an initial parameter $\theta_0 = 2$ and a learning rate $\alpha = 0.1$, calculate the parameter values for the first two iterations (θ_1 and θ_2).

Solution

Step 1: Determine the Gradient and Update Rule. The update rule is $\theta_{new} = \theta_{old} - \alpha \times \nabla J(\theta_{old})$. First, find the derivative (gradient) of the cost function $J(\theta) = 3\theta^2 + 5$:

$$\nabla J(\theta) = 6\theta$$

Step 2: Calculate Iteration 1 (θ_1).

- Compute gradient at $\theta_0 = 2$: $\nabla J(2) = 6(2) = 12$.
- Apply update rule: $\theta_1 = 2 - 0.1(12) = 2 - 1.2 = \mathbf{0.8}$.

Step 3: Calculate Iteration 2 (θ_2).

- Compute gradient at $\theta_1 = 0.8$: $\nabla J(0.8) = 6(0.8) = 4.8$.
- Apply update rule: $\theta_2 = 0.8 - 0.1(4.8) = 0.8 - 0.48 = \mathbf{0.32}$.

Conclusion: The parameter values dynamically descend toward the minimum (which is 0). The values are $\theta_1 = 0.8$ and $\theta_2 = 0.32$.

Question 3.3: Conceptual: Gradient Descent Strategies

You are training a machine learning model on a massive dataset containing 5 million records.

- Why would using standard Batch Gradient Descent be highly problematic in this scenario?
- How does Stochastic Gradient Descent (SGD) solve this specific issue, and what is the trade-off regarding its path to convergence?

Solution

(a) The Problem with Batch Gradient Descent Batch Gradient Descent calculates the gradient of the cost function using the *entire* training dataset before making a single parameter update. For 5 million records, loading all data into memory and computing the average gradient for every single step is extremely computationally expensive and memory-intensive, making the training process unacceptably slow or impossible due to hardware limits.

(b) The SGD Solution and Trade-off Stochastic Gradient Descent (SGD) solves this by updating the model parameters after processing *each individual* training example. This makes iterations lightning-fast and requires very little memory. **Trade-off:** Because it updates based on a single data point's gradient rather than the true global gradient, its path to the minimum is highly noisy and erratic. It experiences high variance, zig-zagging toward the minimum and often oscillating around it rather than settling smoothly.

Question 3.4: Conceptual: The Learning Rate Hyperparameter

In the context of optimization algorithms, describe the geometric and mathematical consequences if the learning rate (α) is set to a value that is far too large.

Solution

The learning rate (α) determines the magnitude of the step taken in the opposite direction of the gradient during optimization. If α is set **too large**:

- Mathematically:** The update calculation ($\theta_{new} = \theta_{old} - \alpha \nabla J$) subtracts a massive term, causing the new parameter to overshoot the optimal value drastically.
- Geometrically:** Instead of stepping down into the "valley" of the cost function, the algorithm jumps completely across the valley to a point higher on the opposite wall.

- **Consequence:** This causes the cost function to bounce back and forth chaotically. It fails to converge and typically diverges, with the error increasing toward infinity as iterations progress.

Question 3.5: Conceptual: L1 vs L2 Regularization

You have a linear regression model with 500 features, but you suspect that only about 15 of these features actually influence the target variable. The rest are just noise. Should you apply Ridge Regression (L2) or Lasso Regression (L1) to improve your model? Explain the mathematical mechanism of your chosen method that makes it suitable for this problem.

Solution

Choice: Lasso Regression (L1) You should apply Lasso Regression.

Mechanism & Justification: Lasso Regression adds a penalty term to the loss function proportional to the *absolute value* of the coefficients ($\lambda * \sum |w|$). A unique mathematical property of the L1 penalty is that it promotes **sparsity**. As the optimization minimizes the loss, the L1 penalty effectively forces the coefficients of less important, noisy features to become *exactly zero*.

Because you suspect only 15 out of 500 features are useful, Lasso acts as an automatic, built-in feature selector. It will isolate the critical features and zero out the 485 irrelevant ones. Ridge Regression (L2), on the other hand, would only shrink all 500 coefficients towards zero but rarely make them exactly zero, retaining the noise in the model.

4 Classification Algorithms (Naive Bayes & Logistic Regression)

Question 4.1: Numerical: Naive Bayes Theorem

A factory uses an acoustic alarm system to predict if a machine is about to fail.

- The prior probability of a machine failing on any given day is 5% ($P(Fail) = 0.05$).
- If a machine is failing, the alarm sounds 90% of the time ($P(Alarm|Fail) = 0.90$).
- If a machine is operating normally, the alarm falsely sounds 10% of the time ($P(Alarm|Normal) = 0.10$).

Using Bayes' theorem and the Law of Total Probability, calculate the probability that the machine is actually failing given that the alarm has sounded, $P(Fail|Alarm)$. Show your steps.

Solution

Step 1: Identify given probabilities.

- Prior $P(Fail) = 0.05$
- Prior $P(Normal) = 1 - 0.05 = 0.95$
- Likelihood $P(Alarm|Fail) = 0.90$
- Likelihood $P(Alarm|Normal) = 0.10$

Step 2: Calculate the Total Probability of an Alarm sounding, $P(Alarm)$.

$$P(Alarm) = [P(Alarm|Fail) \times P(Fail)] + [P(Alarm|Normal) \times P(Normal)]$$

$$P(Alarm) = (0.90 \times 0.05) + (0.10 \times 0.95)$$

$$P(Alarm) = 0.045 + 0.095 = 0.140$$

Step 3: Apply Bayes' Theorem to find the Posterior Probability.

$$P(Fail|Alarm) = \frac{P(Alarm|Fail) \times P(Fail)}{P(Alarm)}$$

$$P(Fail|Alarm) = \frac{0.045}{0.140} \approx \mathbf{0.321} \text{ (or 32.1\%)}$$

Conclusion: If the alarm sounds, there is a 32.1% chance the machine is actually failing. The high false positive rate on the much larger normal population drags the certainty down.

Question 4.2: Numerical: Logistic Regression Sigmoid

A Logistic Regression model is predicting whether a customer will click an ad ($y = 1$). The linear combination of the features and weights (the logit, z) is calculated as $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2$. Given the model parameters: Bias $\beta_0 = -2$, weight $\beta_1 = 3$, and weight $\beta_2 = -1$. For a customer with feature values $x_1 = 2$ and $x_2 = 4$:

- Calculate the linear output z .
- Apply the Sigmoid function to find the predicted probability $P(y = 1|x)$. (Use

$$e \approx 2.718).$$

Solution

(a) Calculate the linear output (z)

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$z = -2 + (3 \times 2) + (-1 \times 4)$$

$$z = -2 + 6 - 4 = 0$$

(b) **Apply the Sigmoid function** The sigmoid function converts the linear logit into a probability between 0 and 1.

$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Substitute $z = 0$:

$$\sigma(0) = \frac{1}{1 + e^0}$$

Since $e^0 = 1$:

$$\sigma(0) = \frac{1}{1 + 1} = \frac{1}{2} = 0.5$$

Conclusion: The model predicts exactly a 50% probability that the customer will click the ad, placing them directly on the standard decision boundary.

Question 4.3: Conceptual: Logistic Regression Limitations

You are using Logistic Regression to classify a chemical compound as "Stable" or "Unstable" based on two sensor readings: Temperature (x_1) and Pressure (x_2). When visualizing the data on a 2D scatter plot, you observe that the "Stable" compounds form a tight cluster in the center, completely surrounded by a ring of "Unstable" compounds.

- Explain why a standard Logistic Regression model will fail to establish a valid decision boundary for this dataset.
- Propose a feature engineering transformation that would allow the Logistic Regression model to separate the classes successfully without switching to a different algorithm.

Solution

(a) **Why Standard Logistic Regression Fails:** Logistic Regression is inherently a linear classifier. It attempts to find a single straight line (or flat hyperplane) to separate the classes. Because the data is distributed in concentric shapes (a center cluster surrounded

by an outer ring), it is geometrically impossible to draw a single straight line that separates the inside from the outside. The model will fail completely.

(b) Feature Transformation Solution: We can overcome this by transforming the coordinate space. Since the classes are separated by their distance from the center, we can engineer a new polynomial feature that captures this circular relationship: Let $z = x_1^2 + x_2^2$. This new feature z represents the squared distance from the origin. The "Stable" compounds in the center will all have low z values, while the "Unstable" ring will have high z values. The Logistic Regression model can now easily learn a simple linear threshold (e.g., $z = \text{threshold}$) in this new 1D space to perfectly separate the classes.

Question 4.4: Numerical: Log Loss Calculation

In Logistic Regression, the Log Loss (Cross-Entropy) function is used instead of Mean Squared Error. For a specific training example, the actual ground-truth label is $y = 0$ (Negative class), but the model confidently predicts a probability of $\hat{y} = 0.95$ for the positive class. Write down the Log Loss formula for a single example, calculate the cost for this specific prediction (use natural log $\ln$), and interpret what this resulting number means regarding the model's performance on this point.

Solution

Step 1: The Log Loss Formula The formula for a single example is:

$$J = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Step 2: Calculation Given $y = 0$ and $\hat{y} = 0.95$: Because $y = 0$, the first term ($y \log(\hat{y})$) becomes 0 and cancels out.

$$\text{Cost} = -[(1 - 0) \log(1 - 0.95)]$$

$$\text{Cost} = -[1 \times \log(0.05)] = -\ln(0.05)$$

Using natural log ($\ln(0.05) \approx -2.995$):

$$\text{Cost} = -(-2.995) = \mathbf{2.995}$$

Step 3: Interpretation A cost of nearly 3.0 is a massive penalty. The true class was 0, but the model predicted there was a 95% chance it was class 1. The Log Loss function heavily penalizes predictions that are both highly confident and entirely wrong, causing a large gradient step to correct this severe error during training.

Question 4.5: Conceptual: Generative vs Discriminative Models

Naive Bayes and Logistic Regression approach classification from two fundamentally different statistical philosophies.

- (a) Identify which algorithm is a Generative Model and which is a Discriminative Model.
- (b) Explain the core difference in what these two philosophies actually attempt to model from the data (using mathematical notation like $P(Y|X)$ and $P(X,Y)$).

Solution

(a) Identification:

- **Naive Bayes:** Generative Model
- **Logistic Regression:** Discriminative Model

(b) Core Difference:

- **Generative Models (Naive Bayes):** These attempt to understand *how* the data was generated. They model the joint probability distribution $P(X,Y)$. They do this by learning the prior probability of a class $P(Y)$ and the likelihood of the features given the class $P(X|Y)$. They essentially learn a complete profile of what each class "looks like."
- **Discriminative Models (Logistic Regression):** These do not care how the data was generated. They directly model the posterior probability $P(Y|X)$. Their only goal is to learn the direct mapping or decision boundary that best separates the classes from one another in the feature space, ignoring the underlying distribution of the inputs.